THREADS

Table of Contents

1. Prerequisites 3

2. Purpose 3

3. Introduction 3

4. Detailed Explanation 4

4.1 Multithreading 4

4.1.1 Real World Scenario 4

4.2 Thread 5

4.2.1 Thread Life Cycle 6

4.2.2 Threads in Java 9

4.2.2.1 Try it Yourself 10

4.2.2.2 Try it Yourself (Example 2) 15

4.2.3 Threads with Thread Class 19

4.2.4 Pros & Cons 20

4.2.5 Check Yourself 21

5. Error Zone - commonly made mistakes 21

6. Sample code 21

7. Additional Information (Optional) 21

8. Best Practices 22

9. Summary 22

10. Related Topics 23

11. References 23

1. Prerequisites

The reader of this topic is supposed to have basic knowledge of Java. An understanding of concurrent programming would be helpful but not necessary.

2. Purpose

In any given application there are lots of tasks (functions) that needs to be performed (executed).There are certain tasks which needs to be performed one after the other, whereas some tasks can be performed simultaneously. A program is always executed sequentially, in the order of the lines of code written in the program. If all the lines of code are executed one after the other in a sequential manner, then the response time of the application would not be as good as compared to code executed in parallel. To execute code in parallel, Java provides the feature of mulch-threading, using which (wherever applicable) we can have the flavor of simultaneously code execution in the application and hence improve the response time of an application.

Multi-threading can be used in scenarios, where same task has to be performed again and again for different kinds of input, or where independent tasks can be executed simultaneously etc. A practical usage of multi-threading can be found in a web server. A web server simultaneously handles several requests (similar or different requests) coming from different users. To get a response from the web server, any user need not wait for other users to complete their requests. Hence with the use of multi-threading web server is able to manage several requests at the same time.

3. Introduction

In our PC, laptop etc., we see a lot of programs running at the same time. This is known as multitasking. When a file is getting downloaded from Internet, the user can read any document. While the user is typing in into a word document, the user can listen to music. While listening to music user can play games. This list goes on. Several different tasks are being performed by the OS at the same time.

What is the advantage to the user?

The answer for this is, user is able to complete multiple tasks within less time. That is, the response time is improved. Wait time for the user is reduced. The user is able to complete multiple tasks in parallel. If the above feature was not available with OS, then for reading a document , the user had to wait till some download from Internet has finished. Similarly the user had to stop preparing a document for listening to music and so on.

Here in this example we see that multiple programs that is a document editor, a media player, a browser, a gaming application etc are running at the same time for better user-experience.

Can we achieve the same simultaneous execution, within one program or application?

Yes, the same kind of simultaneous execution of code can be achieved within one program or application. The multitasking is a feature where different applications execute

simultaneously and they are managed by the OS. The similar parallel execution can be achieved within a program and is known as multi-threading.

Java supports multi-threading feature and we will discuss its details in the upcoming section.

4. Detailed Explanation

4.1 Multithreading

Till now, we have understood that multi-threading is a specialized form of multitasking and in a single application multi-threading, allows simultaneous execution of code, instead of sequential execution.

Lets see one simple application which we come across very often and we are not even aware of that multi-threading is being used.

A text editor, allows formatting of text while the document is being printed. Here in this scenario, two threads are working simultaneously, one is for formatting of the text while the other thread does the job of printing. If multi-threading were absent, then both the tasks could not have been accomplished simultaneously.

So, before moving to the technical details, let us see a small real world example.

4.1.1 Real World Scenario

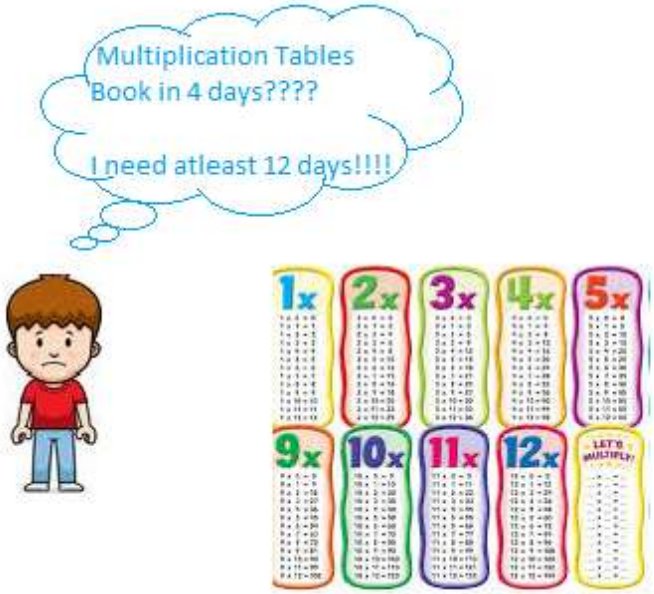
Suppose we have a task to print a multiplication tables book containing multiplication tables from 1 to 12 and this job is given to a primary school student. Let us assume that the student takes one day to prepare one multiplication table page with proper formatting. So in total he would take at least 12 days to complete the book (ignoring binding of the pages etc).

What should we do if the book has to be completed within 4 days? Can this be achieved?

Yes, this can be achieved, if the work is divided between 4 students. Let each student write 3 different multiplication tables. Each student will take 3 days to complete 3 pages. All of them can work in parallel. On the 4th day all the pages can be put together to form the book.

So parallel execution of jobs saved time.

THINK: How much time will it take if 12 students work in parallel (should not we consider the availability of the printing material/printer)? What will happen if more than 12 students work simultaneously.



4.2 Thread

Till this point we did not explain anything about threads, but have been talking about multi-threading. Here let us first clarify what is meant by a thread.

The thread simply means a single line of execution. Or in other words a thread is nothing but a piece of code that is being executed (simultaneously with others).

In Java each program that we execute, executes as a thread. Let us consider a small Java Hello World program. When this program is executed a thread is started which executes sequentially. This thread is called the **Main thread**. Hence each program that is written executes on a single main thread.

In other words, When a Java program starts up, one thread begins running immediately. This is usually called the *main thread* of your program, because it is the one that is executed when your program begins.

If we want simultaneously execution of code, we have to create new threads and have to mention what each thread has to execute. All such created threads execute simultaneously resulting in an efficient program with good response time.

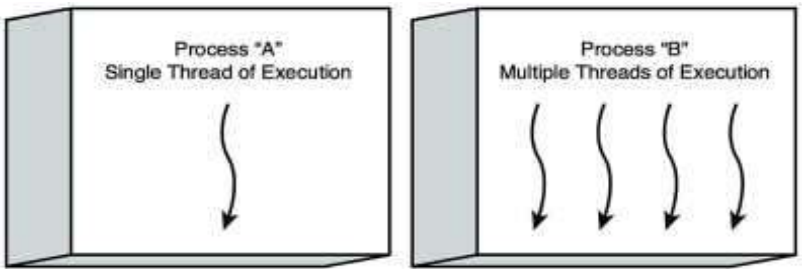


Illustration 1: Single thread and Multi-thread Execution

Details of how to create a thread and execute will be seen in a short while.

NOTE: A program when it is being executed is known as process. Creating a new process is much resource-intensive and processes have their own separate address spaces. Threads are also known as lightweight processes. Threads are created from within a process and share the same memory(address) space as that of the process.

Let us see further details of what different states a thread can be present in.

4.2.1 Thread Life Cycle

Now, when we have got a little idea of what a thread is, the next question that comes to mind is that, *Who creates and manages the threads?*

The programmer writes a program instructing the JVM to create threads. The created threads would be managed by the underlying OS.

The next question is how multiple threads execute simultaneously?

To answer this question, we will assume that we have a single core processor, which can execute one instruction at a time. In this scenario it is not possible to execute multiple

instructions simultaneously. To facilitate simultaneous code execution, the OS, assigns a little time (called time slice) to each of the threads. So the threads take turns for utilizing the processor to execute the code (corresponding to the thread). See the figure below, for better understanding.

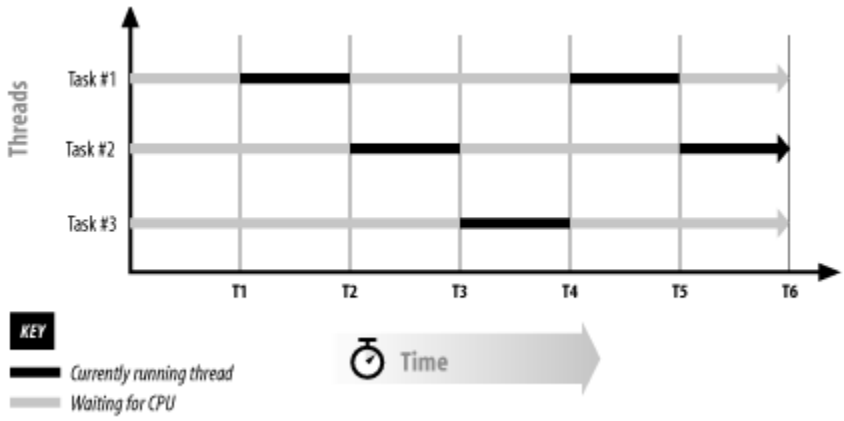


Illustration 2: Thread Scheduling

NOTE: With single core processors, using multi-threading, two or more threads cannot run simultaneously, but the idle CPU time is utilized. In multi-core processors, it is possible for two or more threads to actually execute simultaneously thus providing further efficiency and increase in speed.

There is a Thread scheduler that comes with every OS. This scheduler is responsible for scheduling the threads. Several different algorithms are available for scheduling. The scheduling algorithms assigns a time slice to each thread and executes them one after the other depending on the algorithm. So instead of running one thread completely to the finish, and then executing the next one, each thread will execute few lines of code. This switching of threads is known as the **context switching.**

The next picture (Fig 3 below) depicts the different states a thread can be in and the different transitions that can happen from one state to another state.

The different sates are as follows:

1. **New :** The thread has been created but not yet started.
2. **Runnable:** The thread has been started but not yet scheduled for running. Or the other case could be the time slice allotted for the thread to execute has finished.
3. **Running:** The thread has been picked up by the scheduler and is running .

4. **Non-Runnable(Blocked):** The thread is either waiting for a resource, or sleeping.
5. **Terminated:** The thread has completed its execution and has been terminated.

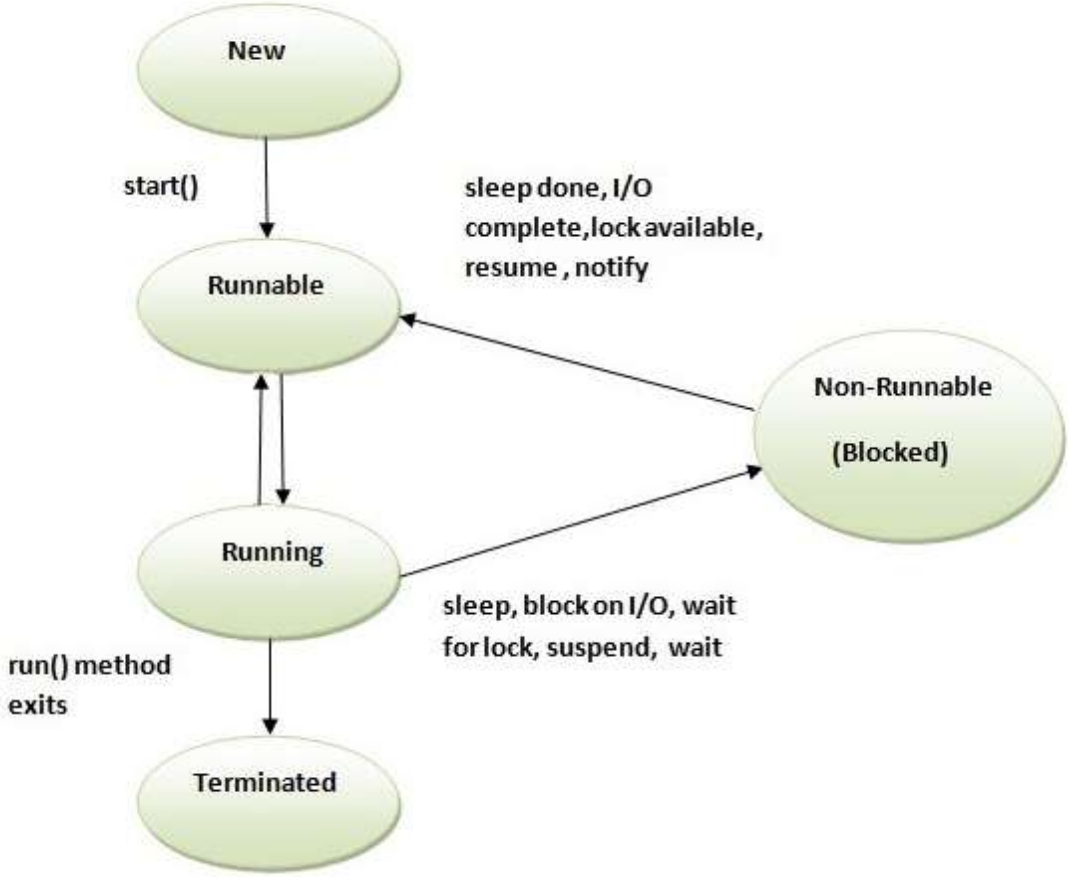


Illustration 3: Thread Life Cycle

When a new thread object is created, it is in *new* state. When the *start()* method is called on the thread (details in a bit later) the thread becomes ready to run and it moves to *runnable* state. *The* OS thread scheduler always picks up threads that are in *runnable* state. Depending on the scheduling algorithm of the OS thread scheduler, the picked thread is assigned a time slot for execution. When the thread is executing in the assigned time slot, then it is known to be in *running* state. Once the time slot is exhausted, the thread again moves to the *runnable* state and waits for its turn to run again. If at any instance a thread requires to wait for other resources, or goes to sleep then it is moved to the blocked state and the next runnable thread is picked up by the scheduler. Thus, when one thread is waiting for a resource, then another thread can complete certain portion of the assigned job resulting in higher CPU utilization and quicker job completion.

4.2.2 Threads in Java

In Java, as soon as a program runs, the Main Thread of the program starts. All the other threads are created and started from the main thread. These threads are known as child threads of the main thread. Often, the main thread has to be the last thread to finish its execution.

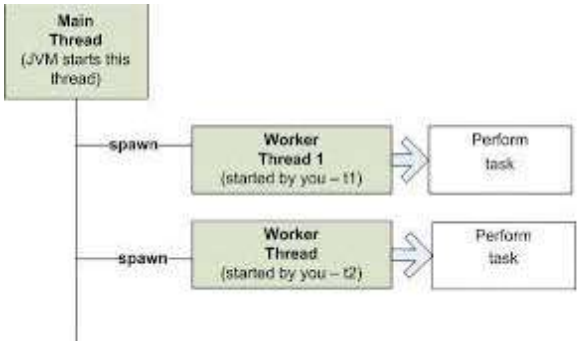


Illustration 4: Main and Child Threads

One point to be noted is that the child threads are also nothing but a piece of Java code and hence they can in turn start other threads (these threads become child of the child threads).

Let us now see the steps that we have to follow to create a thread in Java.

1. A thread can be created either by extending the ***Thread*** class or by implementing the ***Runnable*** interface.
2. In both the cases, the *run()* method has to be overridden. The piece of code that is present in the ***run()*** method would be executed simultaneously by multiple threads.
3. Then the thread has to be started using the ***start()*** method. Now the created threads execute in parallel with main thread.
4. Optionally the main thread can wait for the child threads to finish execution, or can finish its own execution. (But is a good practice for the main thread to wait till the completion of child threads).

Let us see a sample code for implementing the multi-threading feature of Java. Here we are choosing the implementation of thread using Runnable interface as it is preferred over extending Thread class.

IMPORTANT: Implementation of Runnable interface is preferred over extending Thread class for a multi-threaded program. The reason being, if we extend Class A from Thread class, then it

cannot have any other base class hence reducing the flexibility, as well as re-usability. If Class A implements Runnable interface, then it can extend any other class if required and at the same time it is treated as a thread by the JVM (and the underlying OS).

4.2.2.1 Try it Yourself

For the multiplication table book example that we had discussed earlier, let us write a program. We need students, each one of whom would write a page (multiplication table of an assigned number) of the book. In our program we will call the student as PageWriter. There are multiple page writers in our example. So each page writer corresponds to a thread in our program.

1. Hence, the first step is to create a class PageWriter. This class will be used for creating multiple PageWriter objects, or in other words threads that can execute simultaneously.
2. To make this class act like a thread it has to implement the Runnable interface(the preferred approach).
3. The Runnable interface contains an abstract method *run()*. This method has to be overridden in the PageWriter class (because it is implementing the interface).
4. As each PageWriter object has to be assigned with a number corresponding to which it has to print the multiplication table, we have an attribute *mulTableNum* and parameterized constructor for the same.
5. The code present in the run method would be executed simultaneously by multiple threads. Hence, we will write the code for printing a multiplication table for a corresponding number inside the thread.

The complete PageWriter class would be as in the below code snippet:

PageWriter.java:

```java
package com.tcs.ilp.example.thread;

public class PageWriter implements Runnable
{
    private int mulTableNum;

    public PageWriter(int num)
    {
        mulTableNum = num;
    }

    @Override
    public void run()
    {
        for(int i=1; i<=10;i++)
        {
            int mulRes = mulTableNum*i;
            System.out.println(Thread.currentThread().getName()+
                    "is writing "+mulTableNum+" * "+i+" = "+ mulRes);
            try
            {
                //The thread suspends its work for 50 ms.
                Thread.sleep(50);
            } catch (InterruptedException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            }
        }

    }

}
```

NOTE: In the above code snippet, a method *Thread.sleep(50)* is written inside the *run()* method. This method suspends the execution of the thread for the specified time duration (in this case 50 ms, the input parameter to the sleep method). This method can be used to wait for a specified duration, till some condition is satisfied. In this example, this method has been used just to introduce a small delay, so that in the output we can observe multiple threads executing simultaneously.

Similarly the line, *Thread.currentThread().getName()* returns the name of the thread that is assigned to it. If the programmer does not assign any name to a thread, then automatically a name will be assigned to the thread.

1. The next step is to instantiate the PageWriter objects assign them a number and instruct the JVM to treat them as threads, so that they can execute simultaneously.
2. For this let us write a Test class with a main method in it.
3. Create 4 PageWriter objects and assign them 4 different numbers for which multiplication table has to be written.
4. For each of the PageWriter objects, let us assign a name say Writer1, Writer2 and so on.
5. Instruct the JVM that each of the objects have to be treated as threads, by passing the PageWriter object to the Thread constructor. Optionally, the Thread constructor also takes a thread name as input parameter. It can be used to identify threads with name. Now obtain the thread reference.
6. Instruct the JVM to start executing the thread (using the thread reference) by calling the start() method.
7. As soon as the start() method is called corresponding to a PageWriter object the code present in the run() method of that class starts executing in parallel with other threads (including main thread).

Test.java: To test the multithreading program

```java
package com.tcs.ilp.example.thread;

public class Test {

    /**
     * @param args
     */
    public static void main(String[] args)
    {

        System.out.println("Creating multiple threads for writing multiplication tables");

            for(int i=1; i<=4; i++)
            {
                String threadName = "Writer"+i;
                System.out.println("Creating Thread with name "+threadName);
                PageWriter pageWriter = new PageWriter(i); //create an object of PageWriter class

                //Tag the pageWriter object as a thread
                Thread pageWrtThread = new Thread(pageWriter, threadName);

                System.out.println("Starting thread with name "+threadName+
                        "   For multiplication table ::"+(i));
                //Start the thread.
                pageWrtThread.start();
                //This will internally call the run method inside PageWriter class which
                //executes simultaneosly with other threads.

            }

    }

}
```

IMPORTANT: The *run()* method should not be called directly. The *start()* method is an instruction to the JVM, that the thread has to be started, and it internally calls the *run()* method. If the *run()* method is called directly, then JVM will not treat it as a thread and there will be no parallel execution.

Sample Output:

```
Writer2is writing 2 * 1 = 2

Writer4is writing 4 * 1 = 4

Writer3is writing 3 * 2 = 6

Writer1is writing 1 * 2 = 2

Writer4is writing 4 * 2 = 8
```

```
Writer2is writing 2 * 2 = 4

Writer3is writing 3 * 3 = 9

Writer1is writing 1 * 3 = 3

Writer4is writing 4 * 3 = 12

Writer2is writing 2 * 3 = 6

Writer3is writing 3 * 4 = 12

Writer1is writing 1 * 4 = 4

Writer4is writing 4 * 4 = 16

Writer2is writing 2 * 4 = 8

Writer3is writing 3 * 5 = 15

Writer1is writing 1 * 5 = 5

Writer4is writing 4 * 5 = 20

Writer2is writing 2 * 5 = 10

Writer1is writing 1 * 6 = 6

Writer3is writing 3 * 6 = 18

Writer4is writing 4 * 6 = 24

Writer2is writing 2 * 6 = 12

Writer3is writing 3 * 7 = 21

Writer1is writing 1 * 7 = 7

Writer4is writing 4 * 7 = 28
```

4.2.2.2 Try it Yourself (Example 2)

Lets modify the above example, so that once the PageWriter threads finish writing the multiplication tables for a given number, they are assigned with another number for which they have to start writing the multiplication table.

1. The PageWriter class remains same.
2. The main method in the test class has to be modified so that, after the threads finish the work, they are assigned the next number to start working on.
3. For this we will take an array (*pageWriters*) to hold the created references.
4. Modify the main thread so that it waits for the 4 threads (page writers) to finish printing the tables. This is accomplished by using the *join()* method.

5. The join method simply waits for the thread (on which join is called) to finish execution. The main thread will not proceed further the child thread finishes its execution.
6. Once all the threads (page writers) finish execution, pick up the next 4 numbers and assign to the threads. When all the multiplication tables are finished then exit.

The modified Test class is as follows:

Test.java: To test the multithreading program

```java
package com.tcs.ilp.example.thread;

public class Test {

    /**
     * @param args
     */
    public static void main(String[] args)
    {

        System.out.println("Creating multiple threads for writing multiplication tables");
        Thread[] pageWriters = new Thread[4]; //for holding the thread objects

        for(int j=0;j<12; j=j+4)
        {
            for(int i=1; i<=4; i++)
            {
                String threadName = "Writer"+i;
                System.out.println("Creating Thread with name "+threadName);
                PageWriter pageWriter = new PageWriter(i+j); //create an object of PageWriter class

                //Tag the pageWriter object as a thread
                Thread pageWrtThread = new Thread(pageWriter, threadName);

                pageWriters[i-1] = pageWrtThread; //store the reference for further use.

                System.out.println("Starting thread with name "+threadName+
                        "   For multiplication table ::"+(i+j));
                //Start the thread.
                pageWrtThread.start();
                //This will internally call the run method inside PageWriter class which
                //executes simultaneosly with other threads.

            }

           //wait for the above 4 threads to finish.
           for(int i=0;i<4;i++)
           {
               try
               {
                   System.out.println("Waiting for thread ::: "+i+ " to complete");
                   pageWriters[i].join();
               }
               catch (InterruptedException e)
               {
                   // TODO Auto-generated catch block
                   e.printStackTrace();
               }
           }

       }

   }

}
```

Sample Output:

```
Creating multiple threads for writing multiplication tables

Creating Thread with name Writer1

Starting thread with name Writer1   For multiplication table ::1

Creating Thread with name Writer2

Starting thread with name Writer2   For multiplication table ::2

Writer1is writing 1 * 1 = 1

Creating Thread with name Writer3

Starting thread with name Writer3   For multiplication table ::3

Writer2is writing 2 * 1 = 2

Creating Thread with name Writer4

Starting thread with name Writer4   For multiplication table ::4

Writer3is writing 3 * 1 = 3

Waiting for thread ::: 0 to complete

Writer4is writing 4 * 1 = 4

Writer3is writing 3 * 2 = 6

Writer2is writing 2 * 2 = 4

Writer4is writing 4 * 2 = 8

Writer1is writing 1 * 2 = 2

Writer4is writing 4 * 3 = 12

Writer3is writing 3 * 3 = 9

Writer2is writing 2 * 3 = 6

Writer1is writing 1 * 3 = 3

Writer4is writing 4 * 4 = 16

Writer2is writing 2 * 4 = 8

Writer3is writing 3 * 4 = 12

Writer1is writing 1 * 4 = 4

Writer4is writing 4 * 5 = 20

Writer2is writing 2 * 5 = 10

Writer3is writing 3 * 5 = 15

Writer1is writing 1 * 5 = 5

Writer1is writing 1 * 6 = 6
```

```
Writer2is writing 2 * 6 = 12

Writer4is writing 4 * 6 = 24
```

4.2.3 Threads with Thread Class

At the beginning of the section 4.2.2, we had discussed that there are two ways of creating threads. One is by implementing the *Runnable* interface and the other approach is by extending the *Thread* class. In the previous examples we had taken the first and the preferred approach of creating a thread by implementing the *Runnable* interface. Lets see the second approach.

Lets modify the earlier example to create a thread by extending the Thread class. The following changes have to be made to the above example.

1. Make the PageWriter class extend the Thread class instead of implementing the Runnable interface.
2. From the main method, remove the line that creates threads using the Thread constructor.
3. Call the start() method directly using the PageWriter object reference (This is possible because PageWriter class is now extending the Thread class).
4. The other parts of the code remain same.

The below code snippets show the he modifications that have to be made and also the difference between the two approaches of thread programming.

Approach 1

```
public class PageWriter implements Runnable
```

Approach 2

```
public class PageWriter extends Thread
```

Approach 1

```
PageWriter pageWriter = new PageWriter(i+j); //create an object of PageWriter class

//Tag the pageWriter object as a thread
Thread pageWrtThread = new Thread(pageWriter, threadName);

//Start the thread.
pageWrtThread.start();
```

Approach 2

```
PageWriter pageWriter = new PageWriter(i); //create an object of PageWriter class

//Start the thread.
pageWriter.start();
```

4.2.4 Pros & Cons

If multi-threading is used properly and with caution then we can have efficient programs with good response time.

The advantages are:

1. Increase in response time, when compared to single threaded program.
2. Efficient CPU usage. When a thread is blocked or is waiting for I/O, other threads can utilize that CPU time.
3. Efficient utilization of multiprocessor systems.

Challenges with multi-threading programs are:

1. Code becomes more complex
2. Synchronization issues creep in , when multiple threads access the shared resources.
3. Difficulty in testing & debugging, as the code becomes complex, and also it is hard to simulate the bugs.
4. Deadlocks may occur

4.2.5 Check Yourself

1. What is a thread?
2. What is the difference between thread and process?
3. What is the life cycle of thread?
4. How is a thread created?

5. Which is the preferred way of creating a thread?

5. Error Zone - commonly made mistakes

A common beginners mistake is to think that increase in number of threads can actually make the performance of the application better. But this is not true, as there are several factors that need to be considered such as the number of cores the processor has, the other resources available, the number of context switches required etc.

If we go on increasing the number of threads, at one point of time the performance of the application starts degrading.

6. Sample code

7. Additional Information (Optional)

Here in the examples, we have taken threads belonging to one class only. In a practical application there can be multiple threads, belonging to different classes and which execute simultaneously.

The threads can also communicate with each other generally using shared data. In this kind of scenario it might lead to synchronization problems and hence the programmer has to take care that the program handles this problem properly.

Also, JDK 7 adds the Fork/Join Framework, which provides a powerful means of creating multi-threaded applications that automatically scale to make best use of multi-core environments.

Java has a special kind of thread called daemon thread. Following is true about daemon threads

1. Very low priority.
2. Only executes when no other thread of the same program is running.
3. JVM ends the program finishing these threads, when daemon threads are the only threads running in a program.

What are daemon threads used for?

Normally used as service providers for normal threads. Usually have an infinite loop that waits for the service request or performs the tasks of the thread. They can’t do important jobs. (Because we don't know when they are going to have CPU time and they can finish any time if there aren't any other threads running.)

A typical example of these kind of threads is the Java garbage collector.

8. Best Practices

- ✓ Implementing Runnable interface is preferred over extending the Thread class, unless and until there is a need to override the Thread methods.
- ✓ The application should be tested with different number of threads before finalizing the number of threads that should actually execute for better performance.
- ✓ Instead of following the traditional method for creating threads, it is more recommended to use the executor framework.
- ✓ Use the Fork/Join Framework added in JDK 7, which provides a powerful means of creating multithreaded applications that automatically scale to make best use of multi-core environments
- ✓ We need to document clearly how a class behaves during concurrent use.
- ✓ During coding and testing multi-threaded programs we need to keep in mind the synchronization problems that may arise due to parallel execution of code.

9. Summary

Till now we have discussed.

- A thread is a single line or path of execution.
- Each Java program starts with a single Main thread.
- A thread can be created either by
  - implementing the *Runnable* interface or
  - extending the *Thread* class
- Implementing the *Runnable* interface, is the preferred approach to create a thread.
- The *run()* method has to be overridden for creating a thread.
- The thread is started by calling the *start()* method on the thread object.
- The code present in the *run()* method is executed simultaneously with other threads.

10. Related Topics

- Concurrent Programming
- Thread Methods
- Synchronization
- Deadlocks

11. References

Links:

- http://docs.oracle.com/javase/tutorial/essential/concurrency/procthread.html

Books: